SQL 注入漏洞与防护 —— 实验分析报告

任务 1：数据库与查询条件基础说明（3 分）

1、页面中的输入为什么会与数据库查询有关?

2、答：以 DVWA 的用户 ID 查询功能为例，用户输入 ID 后，后端程序会拼接 SQL 语句，向数据库发起查询，将结果返回给页面展示。因此输入内容会直接影响数据库查询逻辑。

2、一个最小业务链路是什么?

答：输入条件 → 数据库查询 → 返回结果。例如：用户输入 1，后端拼接为 SELECT * FROM users WHERE id = '1'，执行后返回用户信息。

3、用户输入是如何进入查询条件的?

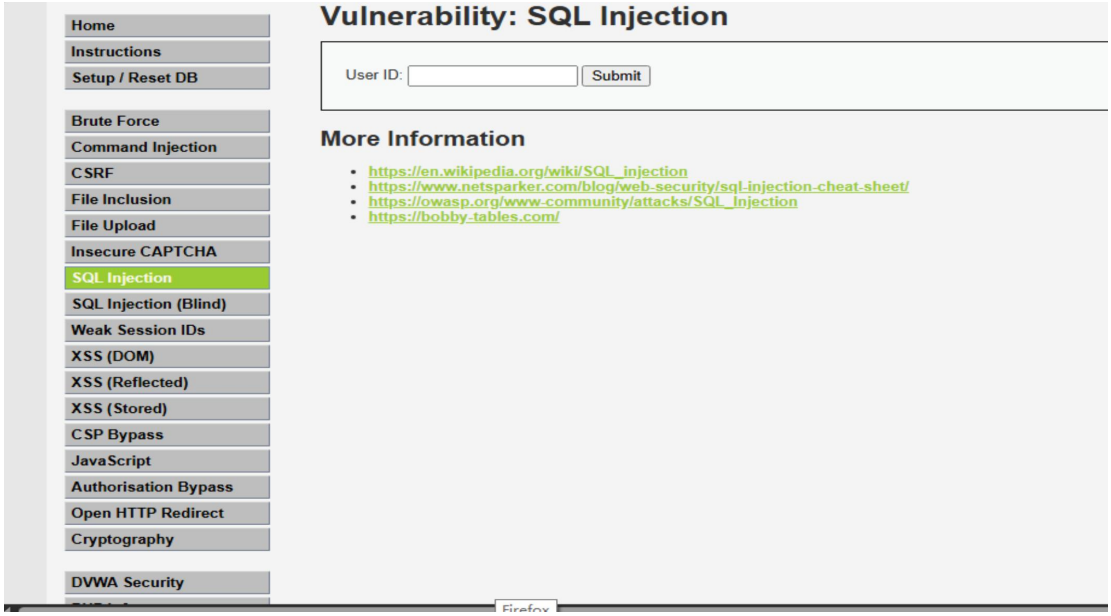
答：用户输入的内容被直接拼接到 SQL 语句中，作为查询条件的一部分。例如输入的 ID 值被直接替换到 SQL 语句的 WHERE 子句中。

4、为什么这为 SQL 注入创造了可能性?

答：当输入内容被直接拼接时，攻击者可以构造特殊字符（如单引号 '、OR 关键字），修改 SQL 语句的结构，使其执行超出预期的逻辑，例如查询所有数据、获取敏感信息甚至执行恶意操作。

任务 2：DVWA SQL Injection Low 现象记录（4 分）

1、

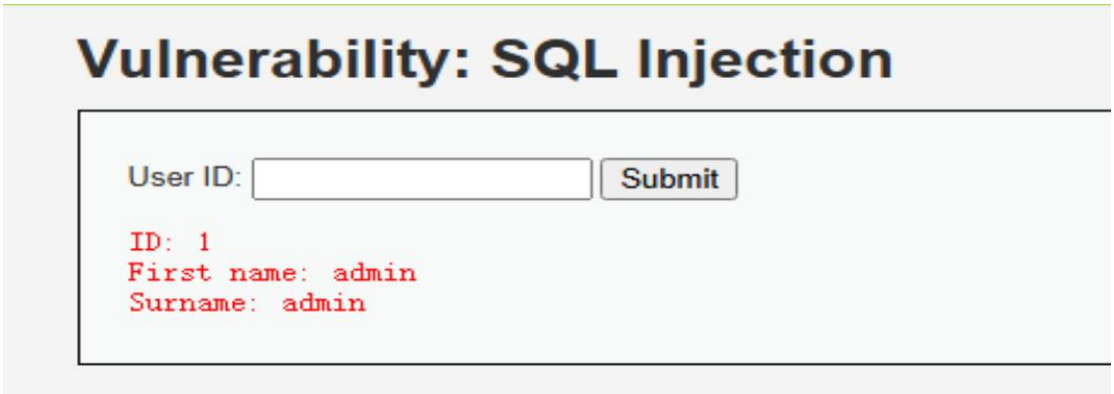


截图内容：DVWA SQL Injection（Low 级别）页面，显示输入框和标题。

分析：该页面为本次 SQL 注入实验的目标环境，用户输入直接用于数据库查询。

2、普通输入与异常输入的对比：

普通输入：

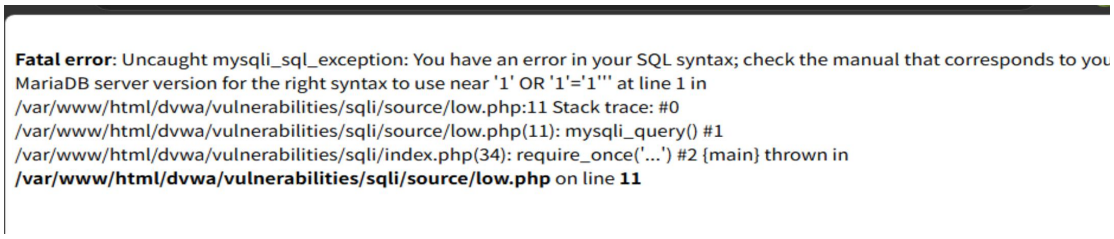


截图内容：输入`1`后，仅返回 ID=1 的用户信息。

分析：这是正常业务查询，SQL 语句为`SELECT * FROM users WHERE id = '1'`，

返回结果符合预期。

异常输入：



Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to you
MariaDB server version for the right syntax to use near '1' OR '1'='1'''' at line 1 in
/var/www/html/dvwa/vulnerabilities/sqli/source/low.php:11 Stack trace: #0
/var/www/html/dvwa/vulnerabilities/sqli/source/low.php(11): mysqli_query() #1
/var/www/html/dvwa/vulnerabilities/sqli/index.php(34): require_once('...') #2 {main} thrown in
/var/www/html/dvwa/vulnerabilities/sqli/source/low.php on line **11**

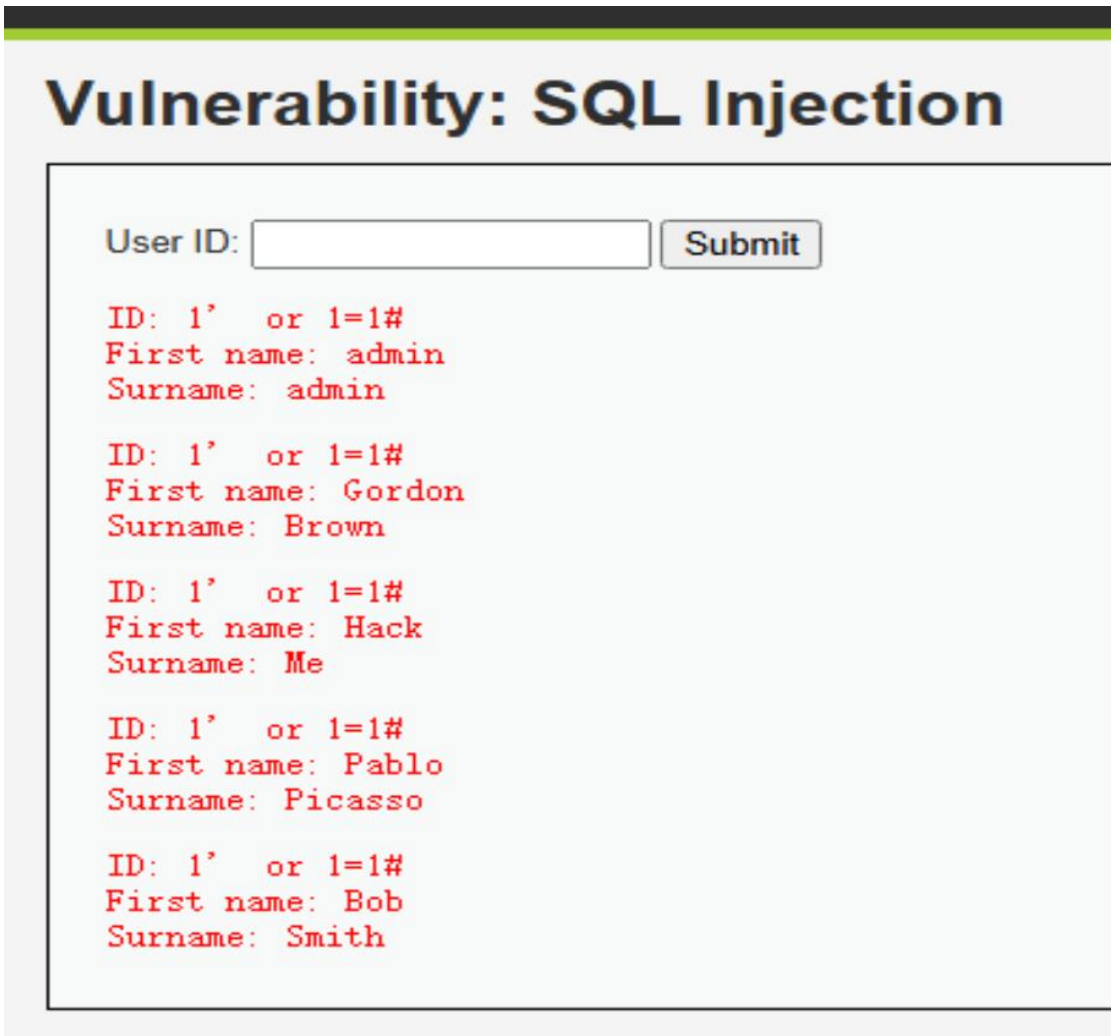
截图内容：输入恶意语句后，返回了所有用户数据。

分析：该输入篡改了 SQL 语句结构，使查询条件变为恒真，返回了全表数据，证明存在 SQL 注入漏洞。

3、返回结果的差异：

普通输入仅返回单条用户数

异常输入返回了全部用户数据



3、为什么这种差异不像 “正常查询变化”？

答：正常的用户 ID 查询只会返回单条结果，而异常输入后返回了所有数据，说明 SQL 语句的逻辑被修改，原本的 WHERE id='1'被拼接为 WHERE id='1' OR '1'='1'，导致条件恒成立，返回所有记录。这不是正常的业务逻辑变化，而是 SQL 语句结构被污染的直接证据。

任务 3：机制分析与差异比较（4.5 分）

1、输入如何进入查询结构?

答：用户输入的内容被后端程序直接拼接进 SQL 语句中，成为查询条件的一部分，改变了原有的 SQL 语句结构。

2、为什么根因是 “结构被污染” 而不是 “字符本身危险”？

答:单引号、OR 等字符本身不危险,危险的是它们被直接拼接进 SQL 语句，修改了查询结构。如果使用参数化查询，这些字符会被当作普通数据处理，不会影响 SQL 结构，也就无法注入。

3、Low 与 Medium 的差异在哪里?

答：Low 级别未对输入做任何过滤，直接拼接 SQL；Medium 级别对输入做了简单的字符转义（如转义单引号），但仍存在其他注入风险，本质上还是依赖黑名单过滤，没有解决结构被污染的核心问题。

4、普通 SQL 注入与 Blind SQL Injection 的差异在哪里?

答：普通注入有直接回显，能通过页面返回的错误信息或数据直接获取注入结果；盲注没有直接回显，只能通过页面的响应差异（如响应时间、内容是否变化）来推断数据，需要通过布尔盲注、时间盲注等方式间接获取信息。

5、为什么盲注和普通注入根因相同，但证据形式不同?

答：两者的根因都是 SQL 语句结构被污染，只是普通注入能直接看到数据差异，而盲注需要通过页面行为差异（如延时、内容变化）来推断注入结果，因此证据形式不同。

强制判断题

1、SQL 注入的根因更接近:

B. 输入进入了不该进入的查询结构

理由: SQL 注入的核心是输入被直接拼接进 SQL 语句，修改了查询结构，而不是输入中的字符本身危险。

2、Blind SQL Injection 和普通 SQL 注入的最大差异更接近:

B.证据形式不同

理由:两者的根因都是结构被污染,差异仅在于普通注入有直接回显,盲注只能通过行为差异推断，因此证据形式不同。

任务 4: SQL 注入防护清单与解释（3.5 分）

防护措施	解决问题	控制层次	不能替代什么
参数化查询	从根本上分离 SQL 语句结构与用户输入，防止输入修改查询逻辑	根因控制（最核心）	不能替代输入校验、错误处理、最小权限等防护
输入校验	限制用户输入的格式、长度、字符范围，过滤危险字符	暴露面控制	不能替代参数化查询，仅靠校验易被绕过
错误处理	关闭数据库详细错误信息回显，防止攻击者获取注入证据	暴露面 / 证据层控制	不能阻止注入发生，只能减少攻击者获取的信息
最小权限	限制数据库用户的权限，即使注入成功，也无法执行高危操作（如删除数据）	后果范围控制	不能阻止注入漏洞本身，只能降低攻击造成的影响
ORM 框架	自动使用参数化查询，避免手动拼接 SQL 语句	根因控制	需正确使用框架，错误的使用方式仍可能产生注入风险

收口项 1：SQL 注入因果链 / 防护关系图

```
graph LR
A[输入点] --> B[输入拼接进SQL结构]
B --> C[数据库解析/执行SQL]
C --> D[结果回显/行为差异]
D --> E[攻击者判断注入成功]
F[参数化查询] -->|阻断| B
G[输入校验] -->|限制| A
H[错误处理] -->|隐藏| D
I[最小权限] -->|限制后果| C
```

收口项 2：学生自评表

1、我本章最清楚的一点是什么？

答：SQL 注入的根因是查询结构被污染，而不是字符本身危险，参数化查询是最核心的防护措施。

2、我最容易混淆的一点是什么？

3、答：容易混淆盲注和普通注入的差异，两者的根因相同，只是证据形式不同。

3、我本次作业中哪一段最需要教师复核？

答：任务 3 中 Low 与 Medium 级别的差异分析部分，希望确认对不同安全级别控制思路的理解是否正确。

4、如果重做一次，我会优先改哪一部分？

答：会补充更多 DVWA 不同级别（如 Medium/High）的注入现象截图，让差异对比更直观。

收口项 3：AI 输出审核与人工复核记录（必做）

1、本作业哪些内容使用过 AI 辅助?

答：SQL 注入防护清单的结构化整理、因果链关系图的绘制、部分理论描述的润色。

2、哪些判断是你自己复核后保留的?

答：SQL 注入根因的判断、普通注入与盲注的差异分析、参数化查询作为核心防护措施的结论，均为复核后保留。

3、哪些内容你发现 AI 输出不准确并进行了修正?

答：AI 最初对盲注和普通注入的根因描述存在偏差，修正为“根因相同，证据形式不同”；同时补充了 DVWA 不同级别的具体场景描述，使分析更贴合实验。

4、你最终如何确认结论和证据是一一对应的?

答：结合 DVWA 实验中的实际输入与页面返回结果，验证了每个理论分析都有对应的现象支撑，例如输入 1' OR '1'='1 后返回所有数据，验证了结构被污染的结论。